# Python Assignment – 01 (Unit-1) (22BCON1068)

DATE : 19/02/24
PAGE NO. : 01.

- Sec-A (5*2=10)

1. What are the main features of Python?

Ans. The main features of Python include:

(i) Simple and easy to Learn
(ii) High-Level Language
(iii) Interpreted
(iv) Object oriented
(v) Cross-Platform
(vi) Open source
(vii) GUI Program
(viii) Dynamic Typed language
(ix) Large standard Library
(x) Easy to code.

2. What is the purpose of an Interactive Shell in python?

Ans. The purpose of an interactive shell in Python, often refferred to as a REPL (Read-Eval-Print-Loop), serves several key functions:

(i) Experimentation and Learning
(ii) Prototyping and Quick Testing
(iii) Interactive Debugging and Troubleshooting
(iv) Script writing and Execution

3. Explain the importance of indentation in python?

Ans. The importance of indentation in Python lies in its role as a fundamental aspect of the language's syntax. it defines the structure of code by visually representing block hierarchy, enforces code blocks, ensure consistency, carries semantic meaning and aids in debugging and maintenance. in essence, proper indentation enhances code readability, maintainability, and clarity, promoting good coding practices and facilitating collaboration among developers.

4. What is the role of identity operators in python?

Ans. Python provides two identity operators: 'is' and 'is not':

Mar 5, 2024, 21:37

These operators are used to ~~Compare~~ Compare the memory locations of two objects rather than their values. The role of identity operators includes:

- Memory Comparison
- Object identity
- Performance optimization
- Distinct from equality

5. Write a Python script that takes a user's name as input and prints a personalized welcome message.

Ans

```
name = input ("Enter your name:")
print ("Welcome," + name "! We're glad to have you here.")
```

- Sec. B (3*7=21)

1. What are the basic features of Python Programming language? Explain the process of editing, saving and running a Python script in a text editor or IDE.

Ans The basic features of the Python language include:

(i) Simple and Readable Syntax: Python has a clean and straightforward syntax, making it easy to understand and write code.

(ii) Interpreted: Python is an interpreted language, which means that code is executed line by line by the Python Interpreter.

(iii) Dynamic Typing: Python is dynamically typed, meaning that variables do not have to be declared with a specific data type.

(iv) High-Level Language: Python is a high-level language, which means that it abstract many low-level details such as memory management, allowing developers to focus on solving problems rather than dealing with implementation details.

(v). Object-Oriented: Python supports object-oriented programming (OOP) concepts such as classes, inheritance, and polymorphism, making it suitable for building large and complex applications.

vi. Cross-Platform: Python code is platform-independent, meaning that if can run on various operating system without modification.

Now, let's discuss the process of editing, saving and running.

(i) Editing:

- Open your preferred text editor or IDE.
- Create a new file or open an existing python script file.

(ii) Saving:

- Once you have written your Python code, save the file with a '.py' extension. Choose a meaningful name for your script to reflect its purpose.

(iii) Running:

- To Run the python script, there are several methods depending on your text editor or IDE:
- IDE Run/Execute Button: Many IDEs a dedicated button or option to run the current python script. Look for a "Run" or "Execute" button in the IDE's toolbar or menu, and click it to run your script.

- Integrated Terminal: Some IDEs have an integrated terminal where you can run python scripts directly. Open the terminal within the IDE, navigate to the directory containing your script, and execute it using the 'python' command as described above.

Mar 5, 2024, 21:37

2. Discuss the history and evolution of Python Programming Language. Illustrate the distinction betn. statements and expressions in python using suitable examples.

Ans. Python's Journey began in the late 1980s under the guidance of Guido Van Rossum in the Netherlands. He envisioned it as a successor to the ABC Language emphasizing code readability and simplicity.

Timeline of its key milestones:

- 1989: Development starts as a project at CWI.
- 1991: First public release, version 0.9.0, introduces core features like classes, functions, and exception handling.
- 2000: Python 2.0 arrives with significants additions like list.
- 2008: Python 3.0 released, making a major revision with improvements in performance and memory managements.
- 2020: Support for python 2 officially ends, with python 3 becoming the standard version.

# Throughout its evolution, Python has consistently focused on:

- Readability - Clear and concise syntax, often described as "more Pythonic" for its natural-language-like structure.
- Simplicity: Fewer lines of code to achieve complex tasks compared to other languages.
- Versatility: Extensive standard library and vast ecosystem of third-party libraries catering

to various domains, including web development, data science and machine Learning.

- Statements vs. Expressions :—

- Statements : These are complete instructions that the interpreter executes, typically ending with a newline or semicolon. They don't directly return a value.

ex:

```
x = 5        # Assignment statement
if x > 3:
    print("x is greater than 3")  # Condition statement.
```

- Expression : These are combinations of variables, operators and function calls that evaluate to a single value. They are used within statements to perform calculations or manipulations.

Ex:

```
result = 2+3*4   # Arithmetic expression
y = max(5,10)    # function call expression
```

# Key difference :

| Statement | Expression |
|---|---|
| (i) A statement executes something. | (i) An expression evaluates to a value. |
| (ii) The execution of a statement changes state. | (ii) The evaluation of a statement does not change state. |

Mar 5, 2024, 21:38

3. (a) WAP in Python that accepts two no. from the user and display their sum, diff", Product and quotient.

```
n1 = int(input("Enter the first number: "))
n2 = int(input("Enter the second number: "))

sum = n1+n2
difference = n1 - n2
Product = n1 * n2

if n2 != 0:
    quotient = n1/n2
else:
    print("Error : Division by zero is not allowed.")

print("The sum of ", n1, " and ", n2, " is ", sum)
print("The differe of ", n1, " and ", n2, " is ", difference)
print("The product of ", n1, " and ", n2, " is ", product)

if n2 != 0:
    print("The quotient of ", n1, " and ", n2, " is ", quotient)
```

# output:

```
Enter the first number : 48
Enter the second number : 25
The sum of 48 and 25 is 73
The diff of 48 and 25 is 23
The product of 48 and 25 is 1200
The quotient of 48 and 25 is 1.92
```

Mar 5, 2024, 21:38

(b) WAP to swap two no. and display value before swapping and after swapping.

Ans

```
n1 = int(input("Enter the first no :"))
n2 = int(input("Enter the second no :"))

print("\nBefore swapping:")
print("Number 1:", n1)
print("Number 2:", n2)

    temp = n1
    n1 = n2
    n2 = temp
print("\nAfter swapping:")
print("Number 1:", n1)
print("Number 2:", n2)
```

# Outputs:

```
Enter the first no: 12

Enter the second no: 18

Before swapping :
Number 1 : 12
Number 2 : 18

After swapping :
Number 1 : 18
Number 2 : 12
```

- Section C (3*11=33)

1. Discuss the concept of operators in python. What does the membership operator do in python and how is it used? How does python determine the order of execution in expressions through precedence and associativity? Provide and example to illustrate this.

Ans: Operators are essential building blocks in any programming language, and python is no exception. They perform various operations on data types like numbers, strings and more. Python supports a variety of operators categorized into different types:

- Arithmetic operators: Perform mathematical calculations (ex:- +, -, *, /)
- Comparison operators: Compare values and return boolean results (e.g:, ==, <, >, !=)
- Logical operators: Combine conditions (e.g:- and, or, not)
- Assignment operators: Assign value to variables (e.g:, =, +=, -=)
- Membership operators: check if a value is present in a sequence (e.g: in, not in)
- Identity operators: Compare objects based on memory location (e.g:, is, isnot)

- Membership Operators (in and not in):- Membership operators are used to check if a value exists within a sequence, such as a list, tuple, string, or set. They evaluate to a boolean value (True or False).

- in: Returns True if the specified value is present in the sequence, False otherwise.

- not in : Returns True if the specified value is not present in the sequence, false otherwise.

ex:

```
numbers = [1,2,3,4]
if 5 in numbers:
    Print ("5 is present in the list.")
else:
    Print ("5 is not present in the list.")
if "a" not in "apple":
    print ("'a' is not present in the string.")
```

- Operator Precedence and Associativity :-

Python uses precedence and associativity rules to determine the order of operations within an expression. Precedence defines which operators are evaluated first, while associativity determines the direction (left to right to left) when evaluating operators with the same precedence.

Here simplified example ~~iss~~ illustrating their combined effects:

Python:

```
expression = 2*3+4/2
```

(i) Precedence : Multiplication (*) and division (/) have higher precedence than addition (+).

(ii) Associativity : Both Multiplication and division have the same left-to-right associativity.

(iii) Therefore, the expression is evaluated as follows:

```
(2*3)+(4/2)
6 + 2
= 8
```

Mar 5, 2024, 21:38

2. Explain the different data types available in Python and provide examples for each. How does type conversion work in Python? Provide examples to demonstrate type conversion in Python.

Ans Python offers several built-in data types for storing and managing different kinds of data. Here's an explanation of the most common ones, with examples:

- Numeric Data Types:
- Int (integers): Used to represent whole numbers.

```
age = 25    # Integer
```

- Float (floating-point numbers): Used to represent numbers with decimal points.

```
pi = 3.14159    # Float
```

- Complex (complex numbers): Used to represent no. with a real and imaginary part.

```
z = 2+3j    # Complex no.
```

- Seaquence Data Types:
- String (str): A seaquence of text characters enclosed in quotes.

ex: name = "Ankit"

- List: An ordered collection of items that can be of different data types. items are mutable (modifiable)

ex:

```
shopping_list = ["bread", "eggs", "milk"]
```

DATE: / /
PAGE NO.: 11

- Tuple: An ordered collection of items that cannot be changed (immutable).

ex.

Coordinates = (10, 20)

- Mapping Type:
- Dictionary (dict): A collection of key-value pairs.

ex. Student = {"name": "Bob", "age": 20, "courses": ["Math", "En"]}

- Boolean Type:
- Bool: Used to represent true or false.

is_adult = True

# Type Conversion: Type conversion allows you to change the data type of a value. basically it is the process of converting one data type into another. Python provides built-in functions to perform type conversion.

Here example:-

(i) Integer to Float:

```
x = 10
float_x = float(x)
print(float_x)
```

output: 10.0

(ii) Float to Integer:

```
y = 3.14
int_y = int(y)
print(int_y)
```

output = 3

Mar 5, 2024, 21:39

(iii) Integer to String :-

```
age = 25
age_str = str(age)
print(age_str)
```

Output: '25'

(iv) String to Integer :

```
num_str = '50'
num_int = int(num_str)
print(num_int)
```

Output: 50

(v) String to Float :

```
pi_str = '3.14'
pi_float = float(pi_str)
print(pi_float)
```

Output: 3.14

(vi) Boolean to Integer :

```
is_student = True
int_student = int(is_student)
print(int_student)
```

Output = 1

- Type Conversion Explicit Casting :-

  - int(x) : Converts x to an integer.
  - float(x) : Converts x to a floating-point no.
  - str(x) : Converts x to a string.

Q3 What will be the output of the following statements if all of them are executed in python interactive mode ?

(a) abs(-2): Returns the absolute value of -2, which is 2. # Output = 2

(b) min(102, 220, 130) :- Returns the minimum value among the given no. because the min() function return smallest of it's arguments.
Output = 102

(c) max(-1, -4, -10) : The max() function return the largest of its arguments.
So Output = -1

(d) max('A', 'B', 'z') : Return the maximum value among the given strings based on their ASCII values.
Output : 'z'. (ASCII value of A = 65, B = 66, z = 90)

(e) max('a', 'B', 'z'): Return the maximum value among the given string based on their ASCII value.

Output : 'a'. (ASCII value of a = 97).

(f) round(1.6) : Rounds the floating-point no. to the nearest integer.
Output : '2'

(g) math.ceil(1.2) : The math.ceil() function rounds a no. up to the nearest integer ceiling. Therefore OUTPUT = '2'.

(h) math.floor(1.8): The math.floor() fun. round a no. down to the nearest integer floor. thus output is 1'

(i) math.log(16,2) : The math.log() fun. calculates the logarithm of a no. (x) to a certain base(base). Here math.log(16,2) is the base-2 log of 16. which is 4.0. so output = 4.0.

(j) math.cos(math.pi) :- The math.cos() fun. Calculates the cosine of an angle in radians.
so math.cos(math.pi) is approximately -1.0.
output = -1.0 (cosine of pi)

(k) print("{0:10,d}".format(12345678)) :- The format() method is used to customize string formatting. in this case, the placeholder {0} is replaced with value {12345678}, and the format specifier :10,d aligns the number to the right with a width of 10 and adds comma separators.
So, the output is " 12,345,678 ".

output : " 12,345,678 ".

the end (22BCON1068)